Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №4
з дисципліни «Технології та методика програмування вбудованих систем»

«Дослідження моделі годинника реального часу»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
ст. викладач, к.т.н Орленко Сергій Петрович

Дата здачі: 29.03.2026

КИЇВ – 2026

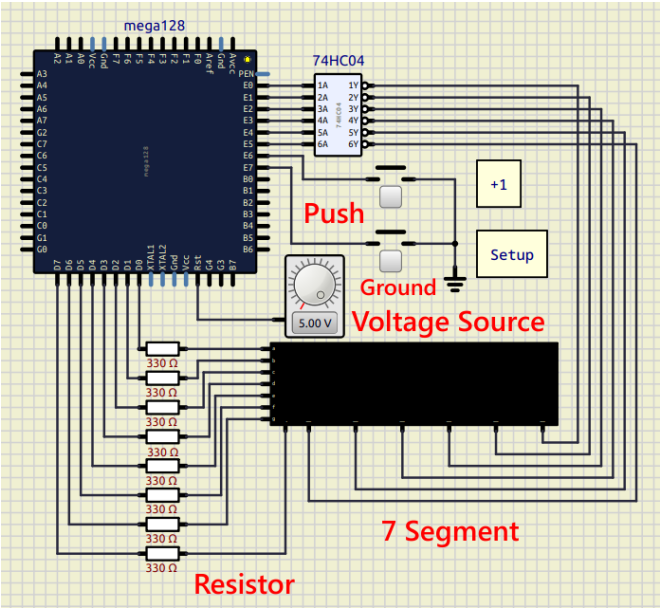
Тема. Моделювання модуля годинника реального часу.

Мета. Користуючись пакетом SimulIDE дослідити моделювання годинника реального часу.

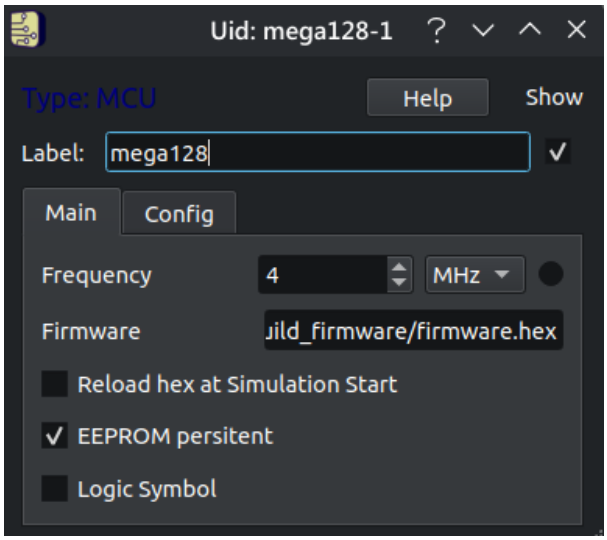
Завдання. Створити модель пристрою в пакеті SimulIDE. Розробити схему алгоритму роботи моделі та робочу програму. Створити hex-файл та підключити його до мікроконтролера. Запустити модель та виконати її дослідження згідно методичних вказівок. Зробити відповідні висновки.

Хід роботи

Виконання лабораторної роботи розпочалося з побудови схеми.



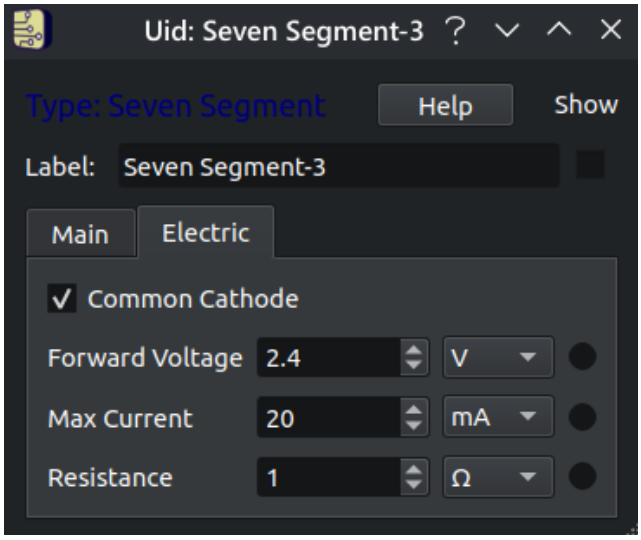
Основним обчислювальним компонентом було обрано мікроконтролер ATmega128. У його властивостях тактову частоту ядра було налаштовано на 4 МГц відповідно до вимог алгоритму.



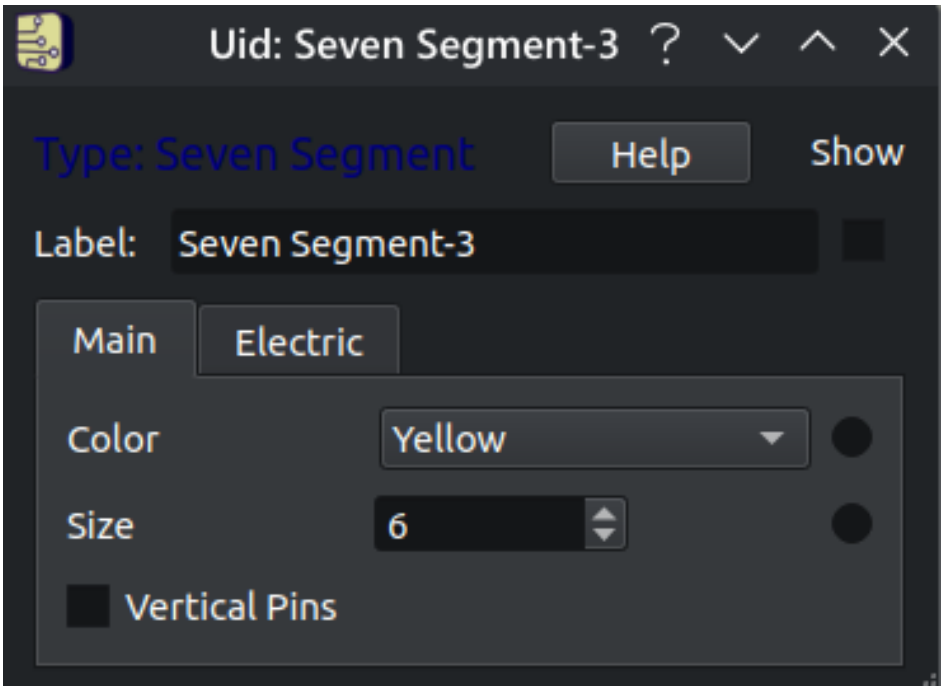
Для забезпечення стабільної роботи без хибних перезавантажень вивід скидання RST був підключений до джерела живлення напругою 5 В.

Відповідно до класичної схеми, для роботи асинхронного таймера потрібен зовнішній годинниковий кварц на 32768 Гц. Проте під час побудови моделі в SimulIDE цей компонент не встановлювався на схему фізично. Це пов'язано з архітектурними особливостями самого симулятора: під час виконання мікроконтролером інструкції встановлення біта `AS0` у регістрі `ASSR`, ядро симуляції автоматично переводить таймер в асинхронний режим, програмно імітуючи наявність необхідного кварцового резонатора.

Для відображення часу було використано мультиплексований 6-розрядний семисегментний індикатор (7-Segment Mux) зі спільним катодом.



Кількість розрядів встановлювалася самостійно.



Спільний катод означає, що для запалювання певного сегмента на його анод потрібно подати логічну одиницю (напругу), а спільний вивід розряду притягнути до землі (логічного нуля). Керування сегментами (піни A-G та крапка DP) здійснювалося через порт D мікроконтролера. Підключення кожного сегмента виконувалося строго послідовно через струмообмежувальні резистори номіналом 330 Ом. Розміщення резисторів саме на лініях сегментів, а не на спільних катодах розрядів, є критично важливим для динамічної індикації: це забезпечує однаковий струм і, відповідно, рівномірну яскравість світіння незалежно від того, скільки сегментів у цифрі світиться одночасно (наприклад, для цифри «1» або «8»).

Вибір активного розряду індикатора здійснювався портом E (піни PE0-PE5). Оскільки керуюча програма генерує на цих пінах логічну одиницю для активації розряду, а індикатору зі спільним катодом потрібен логічний нуль, виникла потреба в інвертуванні сигналів. Замість громіздкої мікросхеми TC4426 та окремих логічних вентилів, для збереження охайності схеми було застосовано мікросхему 74HC04 (Hex Inverter). Вона містить рівно 6 незалежних інверторів, що ідеально відповідає кількості розрядів дисплея. Під час підключення ліній вибору розрядів було враховано порядок виводу даних: найстарший пін PE5 був підключений до першого (найлівішого) розряду, а наймолодший PE0 – до шостого (найправішого). Це усунуло проблему «дзеркального» відображення і забезпечило коректний формат часу ГГ.ХХ.СС.

Керування годинником реалізовано за допомогою двох кнопок: «+1» (підключена до PE6 / INT6) та «Setup» (підключена до PE7 / INT7). Зворотні контакти кнопок з'єднані із загальною лінією землі. Зовнішні підтягуючі резистори не використовувалися, оскільки в коді програмно активовано внутрішні резистори порту E.

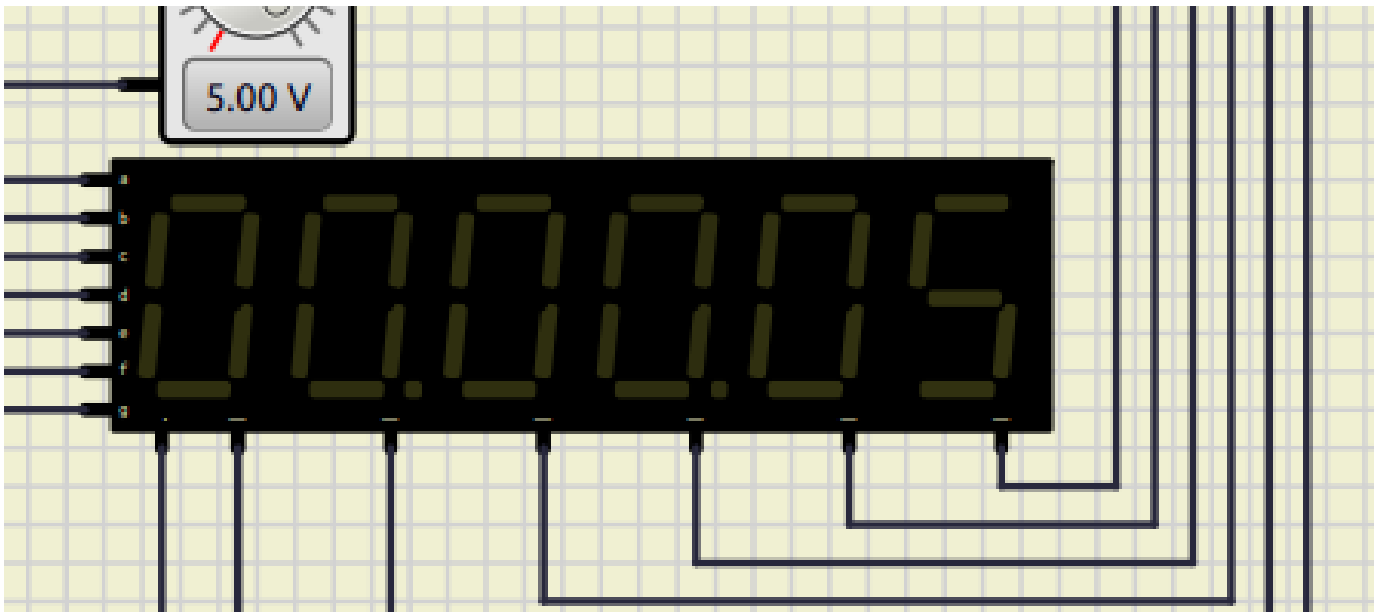
Наступним етапом стала компіляція C-коду за допомогою набору утиліт `avr-gcc`. Під час першого запуску скомпільованої прошивки в SimulIDE виникла критична помилка ядра симуляції: `ERROR: AVR Invalid instruction: ELPM with no RAMPZ`. Ця проблема зумовлена тим, що компілятор `avr-gcc` для мікроконтролерів з пам'яттю понад 64 КБ (як у ATmega128) автоматично генерує стартовий код із використанням розширеної інструкції `ELPM` для копіювання глобальних змінних із флеш-пам'яті в оперативну (сегмент `.data`). Ядро SimulIDE має ваду в обробці цієї інструкції для даної моделі. Для вирішення проблеми алгоритм був модифікований: масиви та глобальні змінні були задекларовані без початкових значень, а їх ініціалізація була перенесена безпосередньо у функцію `init()`. Це дозволило залишити сегмент `.data` порожнім, уникнути генерації інструкції `ELPM` і успішно

запустити симуляцію.

```
-------------------------------------------------------
Executing:
"avr-gcc" -mmcu=atmega128 -Wall -g -Os -o "/home/xairaven/Files/University/
10/Embedded/Lab4/code/build_firmware/"firmware.elf "/home/xairaven/Files/
University/10/Embedded/Lab4/code/firmware.c"

Executing:
"avr-objcopy" -j .text -j .data -O ihex "/home/xairaven/Files/University/10/
Embedded/Lab4/code/build_firmware/"firmware.elf "/home/xairaven/Files/
University/10/Embedded/Lab4/code/build_firmware/"firmware.hex
```

Також до оригінального коду було внесено важливу логічну зміну, пов'язану з відліком часу. У симуляторі SimulIDE для ATmega128 виявлено ігнорування асинхронного режиму таймера T/C0: замість роботи від віртуального кварцу на 32768 Гц, таймер продовжував тактуватися від основної частоти 4 МГц. Через це переривання генерувалися з частотою близько 122 Гц замість 1 Гц, і годинник ішов у 122 рази швидше. Для компенсації цієї апаратної ілюзії симулятора, в обробник переривання `ISR(TIMER0_OVF_vect)` було додано програмний дільник (змінна `sim_divider`), який акумулює 122 виклики переривання перед тим, як здійснити виклик функції інкременту секунд `incSeconds()`. Це нормалізувало хід часу в моделі.



Під час тестування пристрою було успішно перевірено всі режими роботи. При старті мікроконтролер переходить у режим налаштування годин (миготіння перших двох розрядів). Кнопкою «+1» здійснюється інкремент значення (з підтримкою автоінкременту при утриманні). Натискання кнопки «Setup» коректно

перемикає фокус на налаштування хвилин, потім секунд, і врешті-решт переводить годинник у робочий режим, після чого починається точний відлік часу.

Щодо роботи самого програмного коду – він реалізує повноцінний годинник реального часу з динамічною індикацією та можливістю налаштування.

Початкова частина коду відповідає за підключення базових бібліотек для роботи з вводом-виводом та перериваннями мікроконтролерів AVR. Далі йде блок макровизначень, де задаються константи для режимів роботи годинника, границь рахунку для годин, хвилин і секунд у двійково-десятковому форматі, а також позиції цих елементів у масиві часу. Також тут визначається тактова частота процесора, маски для виділення тетрад, параметри для кнопок, налаштування частоти оновлення індикатора та двійкові коди для кожного сегмента дисплея.

Після макросів оголошуються глобальні змінні. Масив для перекодування символів семисегментного індикатора та змінні для керування часом, лічильниками і режимами оголошуються без початкових значень. Це зроблено навмисно, щоб компілятор не створював стартовий код із використанням розширеної інструкції доступу до пам'яті, яка викликає помилку в симуляторі. Всі початкові значення цим змінним присвоюються пізніше, безпосередньо у функції ініціалізації.

Функція інкременту позиції часу є ядром математичної логіки годинника. Оскільки час зберігається у двійково-десятковому форматі, функція перевіряє молодшу тетраду кожного байта, і якщо вона досягає значення десять, додає корегувальну константу для правильного перенесення в старший розряд. Коли значення досягає встановленого ліміту, розряд обнуляється, а функція повертає сигнал про переповнення. Функції інкременту секунд, хвилин та годин використовують цю базову логіку, послідовно викликаючи одна одну при переповненні молодшого розряду.

Логіка виводу інформації зосереджена у функції оновлення цифр на індикаторі. Вона визначає, яку саме цифру з масиву часу потрібно показати зараз, виділяє потрібну половину байта за допомогою зсуву та додає крапку у відповідному місці. Після цього функція гасить індикатор, виводить сформований код символу в порт сегментів і знову вмикає потрібний розряд індикатора, застосовуючи маску миготіння. Після відображення останньої цифри цикл повторюється.

Система працює на базі чотирьох обробників переривань. Обробник переповнення нульового таймера відповідає за відлік реального часу. У ньому реалізовано програмний дільник, який накопичує сто двадцять два виклики перед тим, як додати одну секунду, компенсуючи таким чином особливості симуляції асинхронного режиму. Обробник порівняння першого таймера відповідає за ди-

намічну індикацію, викликаючи функцію оновлення цифр із частотою сто герц. Також він керує лічильником миготіння та обробляє утримання кнопки додавання для активації режиму автоінкременту.

Обробники зовнішніх переривань реагують на натискання кнопок. Переривання сьомого каналу відповідає за кнопку налаштувань і послідовно перемикає режими роботи від редагування годин до звичайного ходу часу. При поверненні в робочий режим ця функція додатково перезапускає нульовий таймер для точного відліку першої секунди. Переривання шостого каналу реагує на кнопку додавання і викликає функцію інкременту того параметра, який редагується в даний момент.

Функції ініціалізації налаштовують усю апаратну частину мікроконтролера. Вони конфігурують порти введення-виведення, задаючи напрямок передачі даних та підтягуючі резистори. Нульовий таймер переводиться в асинхронний режим із відповідним дільником частоти, а перший таймер налаштовується в режим скидання при збігу для генерації стабільної частоти оновлення екрана. Головна функція програми лише викликає загальну ініціалізацію, дозволяє глобальні переривання і переходить у нескінченний порожній цикл, оскільки вся подальша робота пристрою керується виключно апаратними перериваннями.

Висновок. Під час виконання лабораторної роботи було успішно спроєктовано та досліджено модель годинника реального часу на базі мікроконтролера ATmega128 у середовищі SimulIDE. На практиці засвоєно принципи побудови систем динамічної індикації з використанням мультиплексованих семисегментних дисплеїв та мікросхем інвертуючої логіки. Робота з кодом дозволила поглибити знання щодо оптимізації використання пам'яті (уникнення проблеми `ELPM`) та обробки переривань. Розробка програмного дільника частоти продемонструвала гнучкість мікроконтролерних систем у вирішенні апаратних недоліків. У результаті отримано повністю робочу модель, що підтримує налаштування часу та стабільний відлік секунд в асинхронному режимі таймера.

Контрольні запитання.

1. **Для чого може використовуватись таймер/лічильник?**
   Таймери та лічильники використовуються для формування точних часових затримок, підрахунку зовнішніх подій, вимірювання тривалості або частоти вхідних сигналів, генерації сигналів із широтно-імпульсною модуляцією (ШІМ), а також для організації годинника реального часу при роботі в асинхронному режимі.

2. **Які регістри відповідають за дозвіл/заборону переривань від таймерів/лічильників?**

   За дозвіл та заборону переривань від таймерів/лічильників відповідають спеціальні регістри масок переривань TIMSK (для основних таймерів) та ETIMSK (для додаткових таймерів T3, T4, T5 у мікроконтролерах типу ATmega128). Для фактичного спрацювання переривання також має бути встановлений прапорець глобального дозволу переривань у регістрі статусу SREG.

3. **Яким чином задається режим роботи таймера/лічильника `T0` (`T2`)?**

   Режим роботи таймера/лічильника задається за допомогою бітів генерації форми сигналу WGM (Waveform Generation Mode), які розташовані у відповідному керуючому регістрі TCCR0 або TCCR2. Комбінація цих бітів визначає один із доступних режимів, таких як Normal, CTC або різні варіанти ШІМ.

4. **Чим відрізняється режим роботи CTC від режиму Normal?**

   У режимі Normal лічильник завжди рахує до свого максимального апаратного значення, після чого переповнюється, скидається в нуль і починає рахунок знову. У режимі CTC (скидання при збігу) лічильник рахує не до максимуму, а лише до значення, заздалегідь записаного в регістр порівняння OCR, після чого відразу апаратно скидається в нуль, що дозволяє дуже точно налаштовувати частоту переривань.

5. **Які таймери/лічильники можуть працювати в асинхронному режимі?**

   Згідно з характеристиками мікроконтролерів сімейств ATmega64x та ATmega128x, в асинхронному режимі може працювати восьмирозрядний таймер/лічильник T0, оскільки саме він містить спеціальний блок годинника реального часу (RTC). У деяких інших моделях AVR цю функцію може виконувати таймер T2.

6. **Що може виступати в якості задавача частоти при роботі таймера/лічильника в асинхронному режимі?**

   В якості задавача частоти при роботі в асинхронному режимі може виступати годинниковий кварцовий резонатор (зазвичай на 32768 Гц), який підключається до спеціальних виводів мікроконтролера TOSC1 та TOSC2, або ж зовнішній тактовий сигнал від іншої схеми, який подається безпосередньо на вивід TOSC1.

7. **При виникненні яких подій таймери/лічильники T1, T3, T4, T5 можуть генерувати переривання?**

   Шістнадцятирозрядні таймери/лічильники T1, T3, T4 та T5 можуть генерувати переривання при переповненні лічильника (OVF), при збігу значення лічильника зі значеннями у регістрах порівняння каналів A, B або C (COMPA, COMPB, COMPC), а також при захопленні вхідного сигналу на спеціальному піні (CAPT), що дозволяє фіксувати точний час настання зовнішньої події.

8. **За допомогою яких регістрів виконується керування таймером/лічильником?**

   Керування таймером/лічильником виконується за допомогою регістрів керування TCCR (Timer/Counter Control Register), регістрів даних самого лічильника TCNT, регістрів порівняння OCR (Output Compare Register), а також, для конфігурації асинхронного режиму, регістра статусу ASSR (Asynchronous Status Register).

9. **Яким чином здійснюється вибір джерела тактового сигналу, а також запуск і зупинка таймерів/лічильників?**

   Вибір джерела тактового сигналу, а також запуск і зупинка здійснюються за допомогою бітів вибору частоти CS (Clock Select), які знаходяться у керуючому регістрі TCCR відповідного таймера. Якщо в ці біти записати всі нулі, таймер повністю зупиняється, а запис будь-якої іншої комбінації підключає таймер до внутрішнього дільника частоти або зовнішнього джерела і миттєво запускає лічбу.

10. **У чому полягає різниця між роботою 8-розрядних та 16-розрядних таймерів/лічильників у режимі Fast PWM?**

    У 8-розрядних таймерах режим Fast PWM зазвичай використовує фіксоване максимальне значення 0xFF (255) для формування циклу. Натомість 16-розрядні таймери дозволяють налаштовувати апаратну роздільну здатність Fast PWM на 8, 9 або 10 біт, або ж задавати абсолютно довільне максимальне значення через регістр ICR або OCRnA, що дає змогу гнучко змінювати не лише шпаруватість, але й саму частоту ШІМ-сигналу.

Лістинг.

firmware.c

```c
#include <avr/io.h>
#include <avr/interrupt.h>

#define TRUE 1
#define FALSE 0

/* Clock operating modes */
#define MODE_WORK 0
#define MODE_EDIT_SECS 1
#define MODE_EDIT_MINS 2
#define MODE_EDIT_HOURS 3

/* Limits for hours, minutes, and seconds in BCD code */
#define LIMIT_HOURS 0x24
#define LIMIT_MINUTES 0x60
#define LIMIT_SECONDS 0x60

/* Array indices for current time components */
#define I_SECONDS 0
#define I_MINUTES 1
#define I_HOURS 2

/* Number of digits on the display */
#define N_DIGITS 6

/* CPU frequency */
#define F_CPU (4000000UL)

/* Right shifts to move the high nibble to the low nibble */
#define HI_NIBBLE_SHIFT 0x04

/* Mask to extract the low nibble */
#define LO_NIBBLE_MASK 0x0F

/* BCD correction constant when reaching 0xXA */
#define BCD_CORRECTION 0x06

/* Mask for the +1 button pin (PE6) */
#define PLUS_BUTTON_MASK 0x40

/* Delay for holding the +1 button before auto-increment */
#define AUTOINC_DELAY 2

/* Port E initialization mask */
#define PE_INIT_MASK 0x3F

/* Mask to turn off the display (preserves pull-up resistors on PE6, PE7) */
#define INDICATOR_DESELECT_MASK 0xC0

/* Display refresh rate in Hz */
#define REFRESH_RATE 100

/* T/C1 prescaler */
#define PRESCALER 64

/* Display segments */
#define A 1   // 0x01
#define B 2   // 0x02
```

```c
#define C 4   // 0x04
#define D 8   // 0x08
#define E 16  // 0x10
#define F 32  // 0x20
#define G 64  // 0x40
#define DP 128 // 0x80

/* Seven-segment encoding table */
unsigned char LED_CODE[10];

/* T/C1 limit and Blink period as defines */
#define TIMER_MAX (F_CPU / PRESCALER / REFRESH_RATE / N_DIGITS)
#define BLINK_MAX 150

/* Global variables declared without initialization */
unsigned char updateCounter;
unsigned char blinkMask;
unsigned char newBlinkMask;
unsigned char time[3];
unsigned char lastRefreshedDigit;
unsigned char mode;
unsigned char holdCounter;

/* * Increments a specific time component in the array.
* Returns TRUE if overflow occurs.
*/
unsigned char incTimePosition(unsigned char index, unsigned char limit)
{
    if (index < N_DIGITS >> 1) {
        time[index]++;

        /* BCD counting: add 0x06 when reaching 0x0A */
        if ((time[index] & LO_NIBBLE_MASK) == 0x0A) {
            time[index] += BCD_CORRECTION;
        }

        /* Reset to 0 when reaching the limit */
        if (time[index] == limit) {
            time[index] = 0;
            return TRUE;
        }
    }
    return FALSE;
}

/* Refreshes the display digits */
void refreshDigit()
{
    unsigned char index = lastRefreshedDigit >> 1;

    /* Determine if high or low nibble is needed, shifting accordingly */
    unsigned char shift = (lastRefreshedDigit & 0x01) ? HI_NIBBLE_SHIFT : 0x00;

    /* Determine if the decimal point is needed */
    unsigned char point = (shift == 0 && index > 0) ? DP : 0x00;

    /* Turn off the display */
    PORTE &= INDICATOR_DESELECT_MASK;

    /* Output the next digit to Port D */
    PORTD = LED_CODE[(time[index] >> shift) & LO_NIBBLE_MASK] | point;

    /* Select the display position applying the blink mask */
```

```c
    PORTE |= (1 << lastRefreshedDigit) & blinkMask;

    /* Reset after updating the last digit */
    if (++lastRefreshedDigit == N_DIGITS) {
        /* Update the blink mask after a full display cycle to avoid desync */
        blinkMask = newBlinkMask;
        lastRefreshedDigit = 0;
    }
}

/* Increment hours */
void incHours()
{
    incTimePosition(I_HOURS, LIMIT_HOURS);
}

/* Increment minutes */
void incMinutes()
{
    /* Minutes overflowed -> increment hours */
    if (incTimePosition(I_MINUTES, LIMIT_MINUTES)) {
        incHours();
    }
}

/* Increment seconds */
void incSeconds()
{
    /* Seconds overflowed -> increment minutes */
    if (incTimePosition(I_SECONDS, LIMIT_SECONDS)) {
        incMinutes();
    }
}

/* Increment the currently selected time component in setup mode */
void setupIncTime()
{
    if (mode == MODE_EDIT_HOURS) {
        incTimePosition(I_HOURS, LIMIT_HOURS);
    } else if (mode == MODE_EDIT_MINS) {
        incTimePosition(I_MINUTES, LIMIT_MINUTES);
    } else {
        incTimePosition(I_SECONDS, LIMIT_SECONDS);
    }
}

/* Asynchronous timer T/C0 interrupt handler (1 Hz) */
ISR(TIMER0_OVF_vect)
{
    /* SIMULIDE FIX: The simulator ignores async mode and runs T/C0 from the 4MHz main clock.
    * 4,000,000 / 128 (prescaler) / 256 (overflow) = ~122 interrupts per second.
    * We add a software divider to count 122 interrupts before ticking 1 real second.
    */
    static unsigned char sim_divider = 0;

    if (++sim_divider >= 122) {
        sim_divider = 0;
        if (!mode) {
            /* Increment time if not in setup mode */
            incSeconds();
        }
    }
}
```

```c

/* T/C1 interrupt handler (Display multiplexing) */
ISR(TIMER1_COMPA_vect)
{
    /* Refresh a digit on the display */
    refreshDigit();

    if (mode == 0) {
        /* Do nothing else if in normal work mode */
        return;
    }

    /* Increment refresh counter */
    updateCounter++;

    /* Toggle blink state if BLINK_MAX is reached */
    if (updateCounter == BLINK_MAX) {

        /* Is the +1 button pressed? */
        if (PINE & PLUS_BUTTON_MASK) {
            /* Not pressed */
            /* Invert 2 bits in the blink mask depending on the edit mode */
            newBlinkMask ^= ((3 << ((mode - 1) << 1)));

            /* Reset hold counter */
            holdCounter = 0;
        } else {
            /* Pressed */
            /* Start auto-increment if the button is held for AUTOINC_DELAY cycles */
            if (holdCounter == AUTOINC_DELAY) {
                setupIncTime();
            } else {
                holdCounter++;
            }
            /* Disable blinking during auto-increment */
            newBlinkMask = 0xFF;
        }

        /* Reset refresh counter */
        updateCounter = 0;
    }
}

/* Setup button interrupt handler */
ISR(INT7_vect)
{
    if (mode == MODE_WORK) {
        /* Enter hours setup mode */
        mode = MODE_EDIT_HOURS;
    } else {
        /* Move to minutes or seconds setup mode */
        mode--;
    }

    /* Reset blink mask to make all digits visible */
    newBlinkMask = 0xFF;

    /* Reset blink counter */
    updateCounter = 0x00;

    /* Re-initialize async timer upon exiting setup to ensure a full first second */
    if(mode == MODE_WORK) {
        void initAsyncTimer(); // Forward declaration or call
```

```
        initAsyncTimer();
    }
}

/* +1 button interrupt handler */
ISR(INT6_vect)
{
    if (mode == MODE_WORK) {
        return;
    }
    setupIncTime();

    newBlinkMask = 0xFF;
    updateCounter = 0x00;
}

/* Initialize I/O ports */
void initPorts()
{
    /* Set Port D as output */
    DDRD = 0xFF;
    PORTD = 0xFF;

    /* Set PE0-PE5 as outputs, PE6-PE7 as inputs with pull-ups */
    DDRE = PE_INIT_MASK;
    PORTE = 0xFF;
}

/* Initialize asynchronous timer T/C0 */
void initAsyncTimer()
{
    /* Disable T/C0 interrupts */
    TIMSK &= ~((1 << TOIE0) | (1 << OCIE0));

    /* Enable asynchronous mode */
    ASSR |= (1 << AS0);

    /* Reset counter */
    TCNT0 = 0x00;

    /* Set prescaler to 128 */
    TCCR0 |= (1 << CS00) | (1 << CS02);

    /* Wait for registers to update */
    while (ASSR & 0x07);

    /* Enable T/C0 overflow interrupt */
    TIMSK |= (1 << TOIE0);
}

/* Initialize external interrupts */
void initExternalInt()
{
    /* Enable INT6 and INT7 */
    EIMSK |= (1 << INT7) | (1 << INT6);

    /* Trigger interrupts on rising edge */
    EICRB |= (1 << ISC71) | (1 << ISC70) | (1 << ISC61) | (1 << ISC60);
}

/* Initialize T/C1 */
void initTimer1()
{
```

```c
    /* Set CTC mode, prescaler = 64 */
    TCCR1B |= 0x0B;

    /* Reset counter */
    TCNT1 = 0x00;

    /* Set timer compare limit */
    OCR1A = TIMER_MAX;

    /* Enable T/C1 Compare Match A interrupt */
    TIMSK |= (1 << OCIE1A);
}

/* Initialize all peripherals */
void init()
{
    /* Initialize variables here to avoid ELPM with no RAMPZ error */
    updateCounter = 0;
    blinkMask = 0xFF;
    newBlinkMask = 0xFF;
    time[0] = 0;
    time[1] = 0;
    time[2] = 0;
    lastRefreshedDigit = 0;
    mode = MODE_EDIT_HOURS;
    holdCounter = 0;

    LED_CODE[0] = A|B|C|D|E|F;   // 0x3F -> '0'
    LED_CODE[1] = B|C;           // 0x06 -> '1'
    LED_CODE[2] = A|B|D|E|G;     // 0x5B -> '2'
    LED_CODE[3] = A|B|C|D|G;     // 0x4F -> '3'
    LED_CODE[4] = B|C|F|G;       // 0x66 -> '4'
    LED_CODE[5] = A|C|D|F|G;     // 0x6D -> '5'
    LED_CODE[6] = A|C|D|E|F|G;   // 0x7D -> '6'
    LED_CODE[7] = A|B|C;         // 0x07 -> '7'
    LED_CODE[8] = A|B|C|D|E|F|G; // 0x7F -> '8'
    LED_CODE[9] = A|B|C|D|F|G;   // 0x6F -> '9'

    /* Disable Watchdog */
    WDTCR = 0x10;

    initPorts();
    initAsyncTimer();
    initExternalInt();
    initTimer1();
}

/* Main program loop */
int main()
{
    init();

    /* Enable global interrupts */
    sei();

    /* Infinite loop; operation is interrupt-driven */
    while (1);

    return 0;
}
```